

terschiedliche Arten nutzen kann. Dabei sind die unterschiedlichen Möglichkeiten interessant für unterschiedliche Phasen eines Projekts.

Im Zuge der Durchführung dieser Arbeit wird der Quellcode zunächst durch einen Simulator evaluiert. Dieser Schritt ist in Abbildung 3.1 mittig dargestellt. Das Anwendungsprogramm aus dem ersten Schritt wird in den VHDL Quellcode in Form eines Speicherdumps eingebettet, damit der Simulator die Testanwendungen ausführt. Als Simulator kommt GHDL in der Version 5.1.1 zum Einsatz. Die Evaluierung ermöglicht dann die Simulation des Designs für einen bestimmten Zeitschritt, beispielsweise für eine Millisekunde.

Die Simulation wird mit Eingangssignalen "stimuliert". Das Design verarbeitet dann die eingehenden Signale und der Simulator speichert die Signaländerungen in vcd Dateien. Ein sogenannter Waveform-Viewer kann die Signale dann anzeigen wie in 3.2 dargestellt ist.



Abbildung 3.2: Waveforms

In 3.2 lässt sich erkennen, dass die Load-Store-Unit aktiviert wird, da das Signal lsu_en_i auf einen neuen Zustand wechselt. Dann werden Daten geladen und man sieht die 16 einzelnen Rückmeldungen des Datenbus. Durch diese einzelnen Ereignisse kann ein Rückschluss auf die Funktion des Designs gezogen werden.

Durch visuelle Inspektion der Waveform kann nun geprüft werden, ob das Design so auf die Eingabedaten reagiert, wie vom Designer erwartet. Das wichtigste Signal ist die Clocksource, die die CPU taktet, damit die sequentielle Logik Schritte ausführt.

Das NEORV32 Projekt bietet ein vorgefertigtes Skript, welches den Simulator GHDL ausführt. Also solches ist das NEORV32 Projekt für Anfänger sehr gut geeignet, da es diese Funktion bereits automatisch bereitstellt.

Wenn die Simulation die erwarteten Ergebnisse zeigt, dann kann zum finalen Schritt übergegangen werden, der in Abbildung 3.1 auf der rechten Seite dargestellt ist. Das Design wird jetzt synthetisiert statt evaluiert.

Synthese-Werkzeuge führen das sogenannte "Lowering" durch. "Lowering" im Sinne des Wortes soll andeuten, dass die Synthese-Werkzeuge eine Transformation von den abstrakten, also hohen Beschreibungen im Quellcode in Designs aus physischen Logikzellen durchführen. Von abstrakt zu physisch findet also ein "Lowering" statt.

Ein Ausschnitt aus dem Ergebnis der Synthese ist in Abbildung 3.3 zusehen.

In Abbildung 3.3 sind einzelne Look-Up-Tables zusehen, die mit LUT3 bzw. LUT4 beschrieben sind. Die logischen Funktionen aus denen das Design im

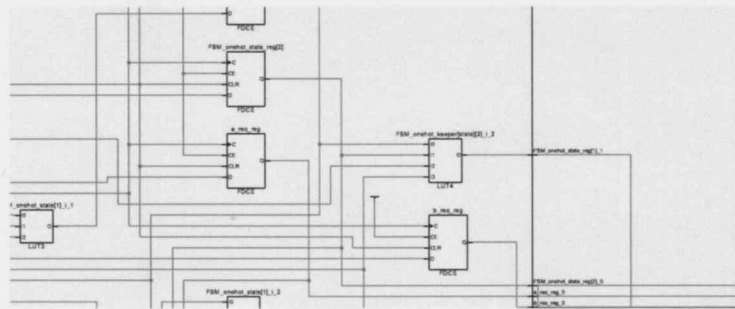


Abbildung 3.3: Schaltplan nach der Synthese

Kern besteht, werden in diesen LUTs abgespeichert und so miteinander kombiniert, dass die Funktion des gesamten Designs realisiert wird. Es ergibt sich ein sehr großes Netz aus Elementen. Ein Mensch kann die Wirkungsweise des Designs auf dieser Detailebene nur sehr schwer verstehen.

Nach der Synthese folgt der Schritt der Implementierung. Die einzelnen LUTs und alle anderen Objekte werden konkret in die Resource eingebettet, die der FPGA bietet! Ein Floor-Plan entsteht. Das Vivado-Synthesewerkzeug kann also tatsächlich grafisch angeben, welche LUTs an welcher Stelle in den FPGA-Chip angeordnet worden sind. Diese ist in Abbildung 3.4 auf der linken Seite dargestellt.

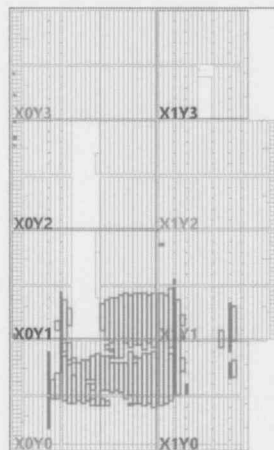


Abbildung 3.4: Implementierung

Sind die Logikzellen bestimmt, lokalisiert und miteinander verbunden, dann kann diese Aufteilung durch eine Bitstream-Datei auf einen FPGA übertragen werden. Dieser Bit-Stream von dem FPGA Chip als Vorgabe verwendet, wie er sich zu konfigurieren hat. Die programmierbare Hardware nimmt jetzt die gewünschte Form an und führt die Funktion des Designs in Hardware aus.

Um die Synthese-Werkzeuge einzusetzen, gibt es ein Parallelprojekt zum NEORV32-Projekt namens, neorv32-setups. Dieses Projekt enthält ein

↑
heim Gedankenstil
sondern Bindestrich
27

Lin
11P
15
1 oder 2 5?
-
Die Implementierung

1 wird LA

TCL-Script, welches in der Entwicklungsumgebung Vivado in der Version 2025.1 ausgeführt werden kann und dann wiederum ein Vivado-Projekt erzeugt. Wenn dieses erzeugte Vivado-Projekt dann schließlich in Vivado 2025.1 geöffnet wird, dann kann der VHDL-Quellcode übersetzt werden.

Dabei führt Vivado seine gesamte Synthese-Toolchain durch und führt damit das Lowering konkret aus. Das Ergebnis ist die Bitstream-Datei. Vivado kann mit dem FPGA Board ARTY A7 100T sprechen und die Bitstream-Datei über eine USB-Verbindung auf den FPGA übertragen. Der FPGA wird sich dem Bitstream zufolge konfigurieren und ab dann läuft die NEORV32 CPU auf dem FPGA, als ob er auf einem Wafer produziert worden wäre.

Erfahrungsgemäß verhält sich das synthetisierte Design anders als das simulierte. Die Arbeit ist also nach der Simulation nicht getan. Die Umgestaltung des Designs nach der Simulation bis zur korrekten Funktion nach der Synthese wird nochmals als eigener Arbeitsschritt bei der Durchführung dieser Arbeit antizipiert. In dieser Phase wird die Funktion über den UART des NEORV32 getestet, der verwendet wird, wenn die Testanwendung einen printf() Aufruf absetzt. Dies ist momentan die einfachste Möglichkeit die Extension zu testen.

3.3 Test-Software

Um zu evaluieren, ob das Design wirklich funktioniert, muss die CPU mit einer Anwendungssoftware betrieben werden, die die neuen Instruktionen wirklich ausführt.

Die Anwendung muss in Form von Maschinencode, also kodierten Befehlen vorliegen. Eine Anweisung ist bei RISC-V eine 32-bit Zahl oder eine 16-bit Zahl, wenn komprimierte Anweisungen unterstützt werden. Die Software kann durch einen Assembler in Maschinencode übersetzt werden. Tatsächlich ist die Kodierung für RISC-V nicht kompliziert und dazu auch gut dokumentiert.

Trotzdem wurde der Einsatz eines Assemblers für das Vorgehen bei dieser Arbeit nicht gewählt, da das Erstellen eines Assembler-Programms ziemlich kompliziert ist. Es bestand die Furcht davor viel Zeit zu verlieren, wenn das Assemblerprogramm angepasst werden muss oder wenn mehr als eine Testsoftware erstellt werden soll. Dann ist für jede Anpassung wieder Assemblerentwicklung notwendig. Direkte Assembler-Entwicklung wird daher nicht verwendet.

Der erste Ansatz an ein lauffähiges Programm zu kommen, war es einen C-Compiler für eine Untermenge der Programmiersprache C zu erstellen. Der Vorteil, der sich aus diesem Ansatz ergibt ist es zum einen volle Kontrolle darüber zu haben, welche RISC-V Anweisungen der Compiler ausgibt. Damit lässt sich ein RISC-V Programm erstellen, das tatsächlich auf der NEORV32 CPU ausgeführt werden kann, wenn man nur Anweisungen generiert, die der NEORV32 auch unterstützt.

- Bindestrihle ver-
- werden

- Bindestrihle
Lit Lm-

- Bindestrihle Lit
Orie?

le

Lr-
- müsste
- solle

LS,
LI, Lt, LV-
LV-

Zum anderen lassen sich in den Compiler dann auch neue Anweisungen einpflegen, die im RISC-V Standard noch nicht definiert sind. Zwangsweise kennt ein GCC Compiler diese Anweisungen folglich auch nicht. Die Erweiterung des GCC Compiler ist um ein vielfaches komplizierter als die Erweiterung eines simplen, selbsterstellten Compilers. Die Unterstützung eigener Anweisungen wird in dieser Arbeit benötigt um die Matrix-Anweisungen verwenden zu können, die im Zuge dieser Arbeit hinzugefügt werden und daher von GCC nicht unterstützt werden.

Da die C Programmiersprache viele syntaktische Elemente und eine Vielzahl an Datentypen enthält, wurde der unterstützte Umfang der Sprache für den eigenen Compiler eingeschränkt. Es gibt nur einen einzigen Datentyp, einen 32 bit Integer. Damit entfällt jegliche Prüfung von Datentypen. Unterstützt werden Funktionsdefinitionen mit Parametern und Aufrufe dieser Funktionen. Dies hat zur Folge, dass der Compiler Stack-Frames erzeugen und auch wieder entfernen können muss. Zusätzlich gibt es for-Schleifen aber keine if-Anweisung, da für eine Matrixmultiplikation mit dem Outer-Produkt keine if-Anweisung benötigt wird. Zur Beschreibung der Matrizen unterstützt der Compiler Arrays vom Integer-Datentyp. Es gibt außerdem einen einfachen Präprozessor, der Define-Anweisungen ersetzen und include-Anweisungen ausführen kann.

Der Compiler erzeugt ein Listing aus Assembler-Befehlen, die dann von einem Assembler in Maschinen-Code übersetzt werden müssen. Es gibt keine Ausgabe von Objekt-Dateien und keine Linker-Infrastruktur und damit auch keine Bibliotheken. Alle Programme müssen komplett in Form von Quellcode vorliegen und können nicht aus vorübersetzten Bibliotheken stammen.

Es wurde ein beträchtlicher Zeitaufwand in diesen Compiler gesteckt und die Teilaufgabe wurde tatsächlich gelöst. Eine Testsoftware zur Matrixmultiplikation lässt sich mit dem Compiler aus der Untermenge von C in RISC-V Assembler übersetzen. Die Erstellung mehrerer Testanwendungen und deren schnelle Anpassung ist effizient möglich.

Zu einem späteren Zeitpunkt wurde zusätzlich auch das Verfahren von Stephan Nolting angewandt. Stephan Nolting hat das Problem von neuartigen Anweisungen auch unter Verwendung des bestehenden GCC Compilers gelöst, indem er Inline-Assembly verwendet um damit Maschinen-Code direkt aus dem Compiler ausgeben zu können. Damit hat er die Möglichkeit geschaffen auch Maschinencode für Anweisungen mit dem GCC zu erstellen, obwohl der GCC diese Anweisungen nicht kennt.

Der Nachteil ist, dass die Verwendung etwas kryptisch ist und viel Erfahrung mit dem GCC voraussetzt. Ein weiterer Nachteil des Inline-Assembly Ansatzes ist, dass der Compiler jetzt nicht mehr autonom arbeiten kann, sondern von einem Menschen gesteuert wird. Damit wird der Compiler davon abgehalten den Quellcode zu optimieren. Für Testfälle ist das Vorgehen zielführend. Für einen industriellen Einsatz muss der Compiler jedoch auf längere Sicht um die Instruktionen erweitert werden um die Optimierung wieder zu ermöglichen.

Das gewählte Vorgehen ist es, die Matrix-Erweiterung mit einer kleinen Test-

Anwendung zu testen, die mit der Strategie von Stephan Nolting und dem GCC übersetzt wird.

17

3.4 Evaluierung der Performance

Die Hardware-beschleunigte Matrix-Berechnung ergibt nur Sinn, wenn sich ein Vorteil gegenüber einer Matrixmultiplikation ohne die Beschleunigung ergibt. Um zu bestimmen, ob sich die Matrix-Erweiterung positiv auswirkt, soll ein Vergleich zwischen zwei Anwendungen durchgeführt werden.

Die Anwendung A berechnet eine Matrixmultiplikation unter Verwendung von drei For-Schleifen. Die Anwendung B führt dieselbe Matrixmultiplikation unter Verwendung der neu hinzugefügten Anweisungen aus. Für beide Anwendungen wird gezählt wie viele CPU-Zyklen die Ausführung benötigt. Es wird selbstverständlich erwartet, dass die Anwendung B die Aufgabe mit weniger CPU-Zyklen erledigt.

Die Voraussetzungen für dieses Vorgehen ist, dass sowohl Anwendung A als auch Anwendung B in den Instruktions-Speicher des NEORV32 passen.

3.5 Zusammenfassung

In diesem Abschnitt wurde beschrieben, welche Schritte zur Durchführung dieser Arbeit durchgeführt werden müssen. Die Planung und Durchführung der Änderungen am NEORV32 Design, das Erstellen von Testsoftware und das Messen des Ergebnisses wurden als wichtige Punkte identifiziert und beschrieben. Es wird zwischen Simulation und Synthese bzw. Ausführung auf dem FPGA unterschieden.

L2-

Zusätzlich wurden die Werkzeuge aufgelistet, mit denen die Entwicklung durchgeführt wird und welche Bedeutung diese Werkzeuge in den einzelnen Schritten haben.

Nachdem das Vorgehen beschrieben ist, werden die Analyseergebnisse am NEORV32 im folgenden Kapitel beschrieben. Darauf folgt ein Kapitel, dass identifizierten Änderungen ausführt, gefolgt von einem Kapitel in dem die Änderungen dann gemessen und ausgewertet werden.

+ 17

+ die
L1,